SLIM: One-shot Quantization and Sparsity with Low-rank Approximation for LLM Weight Compression

Denis Savytski
UC Davis
dsavytski@ucdavis.edu

Yutong Bao
UC Davis
ytbao@ucdavis.edu

Tony Hu
UC Davis
yunhu@ucdavis.edu

Abstract

This report evaluates SLIM through replication and extension experiments on LLaMA-2-13B. Our goal is to verify whether the method's reported advantage under low-bit compression can be reproduced and to examine its sensitivity to different calibration datasets. We compare with dense quantization and Wanda using 4-bit quantization and 50% one-shot pruning. In the replication setting with C4 calibration, we measure zero-shot accuracy on `arc_easy`, `arc_challenge`, `Winogrande`, and `openbookqa`. The reproduced results preserve the main ordering reported in the original paper, Dense > SLIM > Wanda. This confirms that SLIM better maintains model quality under compression than a standard pruning baseline. We further explore Pile and codeparrot as alternative calibration settings to study robustness across different calibration sets. These experiments provide evidence that the main empirical behavior of SLIM is reproducible in practice.

1 Introduction

We reproduce the key results from the paper "SLIM: One-shot Quantization and Sparsity with Low-rank Approximation for LLM Weight Compression" (Frantar et al., 2025). It studies methods for efficiently deploying large language models. Usually, LLMs require high memory and inference costs due to their large parameter counts and the data types they use. The authors solve this problem by developing SLIM, a one-shot model compression method that incorporates 4-bit uniform weight quantization, one-shot pruning, and saliency-based low-rank approximation to reduce inference cost while maintaining high accuracy. Our goal is to familiarize ourselves with the state-of-the-art pruning and quantization methods for LLMs by replicating the experimental results and, if time permits, apply this method in different contexts.

Conventional model compression techniques for LLMs address high memory consumption and slow inference challenges but typically require computationally expensive retraining to preserve accuracy. In contrast, one-shot compression methods eliminate retraining costs, but struggle to achieve accuracy comparable to dense models.This paper presents SLIM, a new one-shot compression framework that holistically integrates hardware-friendly quantization, sparsity, and low-rank approximation into a unified process.

2 Method

2.1 Preliminaries

The central problem addressed by SLIM is how to compress large language models to low precision without incurring the large performance loss that often accompanies standard quantization and pruning. While low-bit compression is attractive for reducing storage and inference cost, it also introduces quantization and clipping errors that may significantly degrade downstream accuracy. SLIM is proposed as a structured solution to this trade-off, aiming to preserve model performance under 4-bit compression and sparse pruning. In our experiments, we investigate whether these claimed benefits can be replicated on LLaMA-2-13B and whether they remain stable across different calibration datasets. Beyond confirming the original results on C4, we also explore Pile and CodeParrot as alternative calibration settings in order to test the efficacy of SLIM under broader natural-language and code-oriented contexts.

2.2 Important definitions/concepts

2.2.1 Quantization

Quantization is the process of replacing higher-precision parameters, such as `float32`, with lower-precision types, such as `int4`. This significantly reduces the memory footprint of the model.

Quantization can be performed in multiple ways; some methods require retraining, while others do not. We focus specifically on methods that do not require retraining. Since we use lower-precision types, we need to somehow recover weights close to their original values; otherwise, model accuracy would degrade significantly.

In the methods we discuss, for each channel we compute a scalar c_i such that

$$\forall j \in \text{Channel}_i \quad c_i \cdot w'_{i,j} \approx w_{i,j}$$

where

- $w'_{i,j}$ is the quantized weight
- $w_{i,j}$ is the original weight before quantization

During inference, the rescaled weights approximate the values the model was trained on, preserving model accuracy.

2.2.2 Pruning

Pruning is the process of zeroing out or removing weights from a model to reduce its memory footprint. Pruning can be done in multiple ways, such as removing weights followed by retraining, or simply setting weights to zero without retraining. In this project we focus on the latter. Pruning introduces sparsity: the proportion of a vector or matrix's entries that are zero. Pruning, therefore, enforces sparsity on the weight matrices. We focus on two paradigms for measuring pruning sparsity:

1. **Unstructured sparsity** — a fixed percentage of the model's weights are set to zero, with no constraint on which weights are chosen.
2. **Semi-structured sparsity** — out of every n consecutive weights, exactly m ($m < n$) are set to zero. For example, in 2:4 sparsity, 2 out of every 4 weights are zeroed out. This pattern is hardware-friendly and supported natively on NVIDIA GPUs.

2.3 Wanda

Wanda (Sun et al., 2024) is a pruning method that we briefly describe here, because it serves both as an important baseline and as a component of the SLiM pipeline.

The core idea is straightforward: run the model on a small calibration set, and for each weight compute an importance score:

$$\text{importance}_{i,j} := |w_{i,j}| \cdot \|x_{i,j}\|$$

where

- $w_{i,j}$ is the weight
- $\|x_{i,j}\|$ is the ℓ_2 norm of the input activations corresponding to that weight, accumulated over the calibration set

Weights with the lowest importance scores are then set to zero, according to the chosen sparsity mechanism (unstructured or semi-structured).

In the original paper, 50% unstructured sparsity on LLaMA-7B resulted in a perplexity increase from 5.68 to 7.26—a moderate degradation with no weight updates required.

In practice, setting weights to zero does not by itself reduce the memory footprint of the model, because this does not change the matrix's shape. Unstructured sparsity also does not provide improvements at inference time, since modern GPU hardware is not optimized for arbitrary sparse patterns. The practical benefit of sparsity is most tangible with **semi-structured (2:4) sparsity**, or structured sparsity where NVIDIA hardware natively supports compressing and skipping the fixed zero pattern, which results in significant speedups. Thus, unstructured sparsity usually works as a baseline in research, and is rarely used in practice.

2.4 SLIM

SLIM's pipeline is divided into 3 sections:

1. SLIM-Quant – quantization of the model
2. Pruning – apply Wanda to prune the quantized weights
3. SLIM-LoRA – use LoRA to reduce the error introduced by quantization and pruning

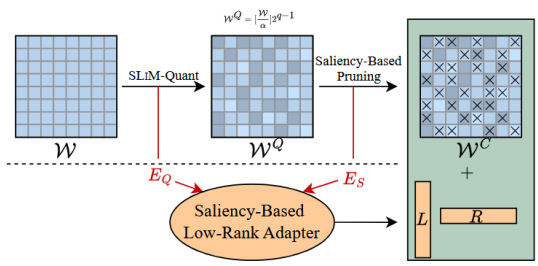

Figure 1: SLIM weight compression pipeline

We now expand on each section.

2.4.1 SLIM-Quant

SLIM uses 4-bit uniform quantization and applies it in two stages: channel rescaling and quantization.

Channel rescaling As established previously, during quantization a single scale parameter is derived per channel. Therefore, if channels operate on significantly different scales, a single scale parameter does a poor job of reconstructing the original weights.

To address this, SLIM rescales channels before quantization: larger channels are downscaled and smaller channels are upscaled by a compensating factor, such that the product of adjacent weight matrices is unchanged. For two consecutive channels C_i and C_j:

$$C_i \cdot C_j = \left(\frac{1}{a} C_i\right)(a C_j)$$

We therefore replace C_i, C_j with $C_i' := \frac{1}{a} C_i$ and $C_j' := a C_j$. The final product is unchanged, but the range across channels is more balanced, reducing quantization error. The rescaling factor a is chosen based on the observed activation statistics of each channel (following the approach of AWQ), and channels are selected for rescaling based on the magnitude of their activations – channels with disproportionately large activations are considered salient and prioritized.

Quantization SLIM-Quant strengthens standard low-bit compression by reformulating the usual quantization objective in a probabilistic way. Under symmetric q-bit quantization, the quantized weight matrix is written as

$$W^Q = \text{round}\left(\text{clip}\left(\frac{W}{\alpha}\right)\right) 2^{q-1},$$

where α is the scaling parameter and $\text{clip}(\cdot)$ truncates values to the interval $[-1, 1]$. The goal is to choose α so as to minimize the weight reconstruction error,

$$\alpha^* = \arg\min_{\alpha} \|W^Q - W\|^2.$$

However, because this objective depends on both rounding and clipping, it is inherently non-convex and does not admit a simple closed-form solution. To address this difficulty, SLIM-Quant replaces the direct matrix-level objective with an expected squared reconstruction error over the empirical weight distribution. In this probabilistic view, the total quantization loss can be decomposed into two interpretable terms: the quantization error for values whose magnitude remains below the clipping threshold α, and the clipping error for values whose magnitude exceeds α and is truncated. As a result, the optimization reduces to choosing α to minimize

$$E_Q(\alpha) = E_{\text{quant}}(\alpha) + E_{\text{clip}}(\alpha).$$

This reformulation makes the scale-selection problem much more tractable and principled than heuristic choices such as AbsMax. Since the true weight distribution is not available in closed form, the paper estimates this objective numerically from the empirical weight histogram and uses a coarse-to-fine multigrid search to efficiently identify the optimal α. In this way, SLIM explicitly balances rounding error and clipping error, which helps preserve model quality more effectively under 4-bit compression.

2.4.2 Pruning

Apply WANDA after quantization.

2.4.3 SLIM-LoRA

Rather than using LoRA for fine-tuning adaptation, SLIM-LoRA repurposes the low-rank structure to partially reconstruct the compression error introduced by quantization and pruning. Let W denote the original weight matrix and W_q the post-quantization and post-pruning weight matrix. The compression error is defined as:

$$E := W - W_q$$

SVD is then performed on E, retaining only the components corresponding to the largest singular values. The key insight motivating this approach is that E is empirically observed to be approximately low-rank, meaning a small number of singular values capture the majority of the error.

One of the main contributions of SLIM-LoRA is that it modifies the SVD decomposition by incorporating saliency into the decomposition. An importance score is calculated on the calibration set, used to reweight the error matrix E prior to applying SVD. Therefore, weights are considered important not simply because of their magnitude, but because of their actual impact on the model output.

The low-rank factors are stored in their decomposed form and the approximation $\hat{E} \approx U_k \Sigma_k V_k^\top$ is added back to the compressed weights at inference time, which yields significant memory savings compared to storing the full error matrix. The rank k – i.e., the number of singular values retained – is a hyperparameter of the framework.

3 Experiments

3.1 Setups

To evaluate how well SLIM performs in our own implementation, we conducted replication experiments to verify its effectiveness under a practical compression setting. We used LLaMA-2-13B as the base model and compared SLIM against two baselines, dense (original model) and Wanda pruning. In the SLIM setting, we applied `50%` sparsity, and 4-bit quantization. For replication, we used C4 dataset , web-text calibration mentioned in the original paper. We evaluated the compressed models using zero-shot accuracy on the selected benchmark tasks.

Throughout this experiment, two benchmark runs collapsed during evaluation, but we were still able to report results on 4 out of the 6 tasks used in the original paper:

- `arc_easy` - contains easier science multiple-choice questions;
- `arc_challenge` - contains more difficult science reasoning questions;
- `winogrande` - a commonsense reasoning task focused on choosing the correct reference in a sentence;
- `openbookqa` - a science question answering benchmark that requires both facts and reasoning.

Through this setup, our goal was to compare the overall accuracy and task-level behavior of the different compression schemes.

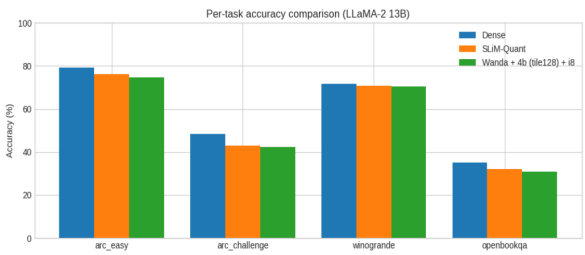

Figure 2: Our results for reproducing the paper

3.2 Confirming on Original Paper

Our replication results provide a clear confirmation of the main empirical findings reported in the SLIM paper. Across the four benchmark tasks that completed successfully, the overall ranking of methods remains consistent across all datasets: the Dense model achieves the highest average zero-shot accuracy, SLIM performs second, and Wanda performs third. In our reproduced results, the average accuracies are 58.54% for Dense, 55.52% for SLIM, and 54.61% for Wanda. This same ordering also appears at the task level, where SLIM consistently stays close to the Dense baseline while maintaining an advantage over Wanda. In particular, the reproduced results show that SLIM preserves more accuracy than a standard pruning baseline after compression, which matches the central claim of the original work.

When compared directly with the paper, our reproduced average accuracies are slightly lower for all three methods, but the differences are small and stable. The reproduced Dense, Wanda, and SLIM results all fall within a narrow margin of roughly two to three percentage points below the paper's reported values, while still preserving the same relative ordering between methods. This suggests that our implementation successfully captures the core behavior of the method even though the absolute scores do not match exactly. A likely reason for the gap is that our evaluation was conducted on only 4 of the 6 benchmarks, since two runs collapsed during testing. Overall, these results support the conclusion that our replication is faithful to the original paper and that the main advantage of SLIM over Wanda is reproducible in practice.

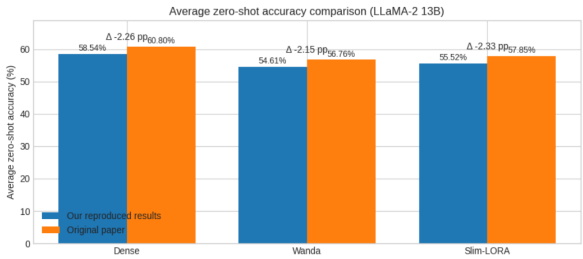

Figure 3: Our results vs paper

3.3 Extended Studies on Pile and Codeparrot

The original paper does not perform a study on calibration sets. In this section, we aim to evaluate the difference in performance based on calibration sets:

- C4 - a large web text dataset used in the original paper
- Pile - a diverse mixture of natural language sources

Table 1: Average zero-shot accuracy of SLIM for LLaMA-2 13B

Method	Calib. Set	Average	ARC-Easy	ARC-Challenge	Winogrande	OpenBookQA
Dense		58.54	79.21	48.29	71.66	35
Wanda	C4	54.61	74.70	42.32	**70.87**	**32.00**
Wanda	Pile	54.34	74.07	41.38	70.72	31.20
Wanda	Codeparrot	54.34	74.07	41.38	70.71	31.20
SLIM	C4	**55.52**	**76.30**	**42.91**	**70.87**	31.20
SLIM	Pile	54.30	74.58	41.38	69.85	31.40
SLIM	Codeparrot	54.30	74.58	41.38	68.85	31.40

- CodeParrot - a dataset of programming language data

We select these datasets to cover different datasets. C4 serves as the baseline setting used in the original work. The Pile provides a broader and more diverse distribution of natural language, allowing us to evaluate whether increased domain diversity affects compression performance. CodeParrot consists of code data and represents a distribution that differs significantly from standard natural language. Together, these datasets enable us to study how the choice of calibration data influences the behavior of different compression methods across domains.

The experiments show that the three compression schemes perform very similarly overall, with only a narrow spread in average zero-shot accuracy. Among them, SLIM on C4 achieves the best average score at **55.52%**, slightly above Wanda (Pile) at **54.34%** and SLIM (Pile) at **54.30%**. At the task level, SLIM leads on all four reported benchmarks: `arc_easy` (**76.30%**), `arc_challenge` (**42.92%**), `Winogrande` (**70.88%**), and `OpenBookQA` (**32.00%**), although the margins are small in some cases. The closest comparison appears on `Winogrande`, where all three methods remain near **70%**, suggesting that this task is relatively stable under different compression choices. By contrast, the gap is somewhat more visible on `arc_easy` and `arc_challenge`, where SLIM keeps a modest but consistent advantage. Overall, the Pile-based calibration results suggest that SLIM remains the strongest of the three methods in this setting, while the small differences also indicate that all methods behave comparably when calibrated on a broad and diverse text distribution.

In the additional calibration experiments using CodeParrot subsets, the resulting benchmark scores were found to be extremely close as Pile. While this could suggest that SLIM is not highly sensitive to the choice of calibration domain, our earlier debugging indicates that the similarity is more likely due to an implementation issue rather than a true empirical equivalence. Specifically, some experiment outputs were still being written to Pile-associated save paths, and the reuse of an unchanged `RUN_TAG` or checkpoint configuration may have caused the CodeParrot run to load or overwrite results from the previous Pile experiment. Therefore, these two calibration settings cannot yet be treated as fully independent evidence, and the observed similarity should be viewed as tentative.

4 Related Work

Pruning and One-shot Compression Post-training compression methods aim to reduce model size without retraining. Among these, Wanda (Sun et al., 2024) has emerged as a strong baseline for one-shot pruning of large language models. It is widely used in subsequent work due to its simplicity and effectiveness. In this project, we use Wanda as both a baseline and as a component within the SLIM pipeline.

Unified Compression Methods Recent approaches attempt to combine multiple compression techniques into a single framework. SLIM (Frantar et al., 2025) is a representative example that combines quantization, pruning, and low-rank approximation in a unified pipeline. It jointly applies them in a one-shot setting. Our work focuses on reproducing this method and validating its empirical behavior.

Calibration Data Effects A key component of post-training compression is the use of calibration

data to estimate activation statistics. Prior work (e.g., Lin et al., 2023) shows that the choice of calibration set can significantly influence quantization quality. However, the effect of calibration data on unified pipelines such as SLIM is not well understood. To address this, we evaluate multiple calibration datasets (C4, Pile, and CodeParrot) and analyze their impact on performance.

5 Conclusion

We reproduced the key results from the paper "SLIM: One-shot Quantization and Sparsity with Low-rank Approximation for LLM Weight Compression" (Frantar et al., 2025) and found that the main experimental trend in the paper can be replicated in our implementation. In our results, the Dense model achieves the highest accuracy, SLIM performs second, and Wanda performs third, which is consistent with the paper. We experienced failures on 2 out of 6 datasets used in the original paper; therefore, the presented results only include 4 datasets, which explains why our accuracy scores are slightly lower than those reported in the original paper.

Our experiments also show that SLIM provides a strong balance between compression and accuracy. Compared with Wanda, SLIM preserves more zero-shot accuracy after compression while still using a one-shot pipeline without retraining.

In addition to reproducing the original paper, we also carried out extra experiments using Pile and CodeParrot as calibration sets. These experiments were not included in the original paper, but they helped us further explore how SLIM behaves under different calibration settings. The additional Pile experiment took a long time to run even on an H100 GPU and did not outperform Wanda, so it did not show a clear advantage in our implementation. The CodeParrot experiment remains preliminary, as potential implementation issues may have influenced the reported results.

References

Elias Frantar, Saleh Ashkboos, and Dan Alistarh. 2025. Slim: One-shot quantization and sparsity with low-rank approximation for llm weight compression. In *International Conference on Machine Learning (ICML)*.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations (ICLR)*.